

smart-build 独立构建系统技术实施方案

项目目标

将 smart-build 项目从 yocto/poky 依赖中解耦，使其成为一个独立的构建系统，仅依赖于 bitbake。

目标架构

```
smart-build/
├── bitbake/                # BitBake 构建引擎 (git submodule)
├── meta-core/              # 核心基础层 (替代 poky/meta)
│   ├── conf/
│   │   ├── layer.conf
│   │   ├── bitbake.conf
│   │   └── machine/
│   │       ├── qemuarm64.conf
│   │       └── qemuarm64.conf
│   │       └── distro/
│   │           └── smart.conf
│   └── classes/
├── meta-rt-smart/          # RT-Smart 专用层 (现有)
├── smart-env               # 环境初始化脚本
└── build/                  # 构建目录 (动态生成)
    ├── conf/
    │   ├── local.conf
    │   └── bblayers.conf
```

分阶段实施步骤

阶段一：环境准备 (1-2天)

步骤 1.1：添加 BitBake 子模块

```
git submodule add https://github.com/openembedded/bitbake.git
git submodule update --init --recursive
```

步骤 1.2：分析依赖关系

- 分析 meta-rt-smart 对 poky/meta 的实际依赖
- 确定最小化核心组件清单
- 制定组件迁移计划

阶段二：创建核心基础层 (2-3天)

步骤 2.1：创建目录结构

```
mkdir -p meta-core/{conf/{machine,distro},classes,recipes-core}
```

步骤 2.2：创建核心配置文件

meta-core/conf/layer.conf

```
BBPATH .= ":${LAYERDIR}"
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb ${LAYERDIR}/recipes-*/*/*.bbappend"
BBFILE_COLLECTIONS += "core"
BBFILE_PATTERN_core = "^${LAYERDIR}/"
BBFILE_PRIORITY_core = "5"
LAYERSERIES_COMPAT_core = "styhead"
```

meta-core/conf/bitbake.conf

```
# 基础构建配置
BUILD_ARCH = "${@os.uname()[4]}"
BUILD_OS = "${@os.uname()[0].lower()}"
BUILD_SYS = "${BUILD_ARCH}-${BUILD_OS}"

# 工作目录配置
TMPDIR ?= "${TOPDIR}/tmp"
WORKDIR = "${TMPDIR}/work/${MULTIMACH_TARGET_SYS}/${PN}/${PV}-${PR}"
S = "${WORKDIR}/${BP}"
B = "${S}"

# 目标系统设置
TARGET_ARCH ?= "${MACHINE_ARCH}"
TARGET_OS = "linux"
TARGET_SYS = "${TARGET_ARCH}-${TARGET_OS}"
```

步骤 2.3: 创建机器配置

meta-core/conf/machine/qemuarm64.conf

```
#@TYPE: Machine
#@NAME: QEMU ARMv8 machine
#@DESCRIPTION: Machine configuration for ARMv8 on QEMU

MACHINE_ARCH = "aarch64"
DEFAULTTUNE = "aarch64"

# RT-Smart specific settings
RTSMART_BSP = "qemu-virt64-aarch64"
RTSMART_ARCH = "aarch64"
RTSMART_TOOLCHAIN = "aarch64-linux-musleabi"
```

meta-core/conf/machine/qemuriscv64.conf

```
#@TYPE: Machine
#@NAME: QEMU RISC-V 64-bit machine
#@DESCRIPTION: Machine configuration for RISC-V 64-bit on QEMU

MACHINE_ARCH = "riscv64"
DEFAULTTUNE = "riscv64"

# RT-Smart specific settings
RTSMART_BSP = "qemu-virt64-riscv"
RTSMART_ARCH = "riscv64"
RTSMART_TOOLCHAIN = "riscv64-linux-musleabi"
```

步骤 2.4: 创建发行版配置

meta-core/conf/distro/smart.conf

```
#@TYPE: Distribution
#@NAME: Smart
#@DESCRIPTION: RT-Smart Distribution

DISTRO = "smart"
DISTRO_NAME = "RT-Smart"
DISTRO_VERSION = "1.0"

# Feature settings
DISTRO_FEATURES = "largefile multiarch"
PACKAGE_CLASSES ?= "package_tar"
INIT_MANAGER = "none"
```

阶段三: 创建环境脚本 (1天)

步骤 3.1: 创建 smart-env 脚本

smart-env

```
#!/bin/bash
# smart-env - RT-Smart Build Environment Setup Script
# Usage: source smart-env [build_dir]

if [ -n "$BASH_SOURCE" ]; then
    THIS_SCRIPT=$BASH_SOURCE
elif [ -n "$ZSH_NAME" ]; then
    THIS_SCRIPT=$0
else
    THIS_SCRIPT="$(pwd)/smart-env"
fi

SMART_BUILD_ROOT=$(cd $(dirname $THIS_SCRIPT) && pwd)
BUILD_DIR=${1:-build}

# 创建构建目录
if [ ! -d "$BUILD_DIR" ]; then
    mkdir -p "$BUILD_DIR/conf"
    echo "Created build directory: $BUILD_DIR"
fi
```

```

# 设置环境变量
export SMART_BUILD_ROOT
export BBPATH="$SMART_BUILD_ROOT/meta-core:$SMART_BUILD_ROOT/meta-rt-smart"
export BB_ENV_EXTRAWHITE="SMART_BUILD_ROOT MACHINE DISTRO"
export PATH="$SMART_BUILD_ROOT/bitbake/bin:$PATH"

cd "$BUILD_DIR"

# 创建 local.conf
if [ ! -f conf/local.conf ]; then
    cat > conf/local.conf << 'EOF'
# RT-Smart Build Configuration
MACHINE ??= "qemuarm64"
DISTRO ?= "smart"

# Build directories
TMPDIR = "${TOPDIR}/tmp"
DL_DIR ?= "${TOPDIR}/downloads"
SSTATE_DIR ?= "${TOPDIR}/sstate-cache"

# Parallelism Options
BB_NUMBER_THREADS ?= "${@oe.utils.cpu_count()}"
PARALLEL_MAKE ?= "-j ${@oe.utils.cpu_count()}"

# Package Management
PACKAGE_CLASSES ?= "package_tar"
EOF
    echo "Created conf/local.conf"
fi

# 创建 bblayers.conf
if [ ! -f conf/bblayers.conf ]; then
    cat > conf/bblayers.conf << EOF
# Layer Configuration
BBPATH = "\${TOPDIR}"
BBFILES ?= ""

BBLAYERS ?= " \\\
    $SMART_BUILD_ROOT/meta-core \\\
    $SMART_BUILD_ROOT/meta-rt-smart \\\
"
EOF
    echo "Created conf/bblayers.conf"
fi

echo ""
echo "### RT-Smart Build Environment Ready ###"
echo "Build Directory: $(pwd)"
echo "Machine: ${MACHINE:-qemuarm64}"
echo "Distribution: ${DISTRO:-smart}"
echo ""
echo "Common build targets:"
echo "  bitbake smart-gcc -c install_toolchain"
echo "  bitbake rt-smart -c build_all"

```

阶段四：适配现有配方 (1-2天)

步骤 4.1: 更新 meta-rt-smart 依赖

```
# 更新 meta-rt-smart/conf/layer.conf
LAYERDEPENDS_meta-rt-smart = "core"
```

步骤 4.2: 创建基础类文件

meta-core/classes/base.bbclass

```
# 基础构建类
BB_DEFAULT_TASK ?= "build"

addtask build
do_build[noexec] = "1"

python base_do_unpack() {
    src_uri = (d.getVar('SRC_URI') or "").split()
    if not src_uri:
        return

    fetcher = bb.fetch2.Fetch(src_uri, d)
    fetcher.unpack(d.getVar('WORKDIR'))
}

addtask unpack after do_fetch
do_unpack[dirs] = "${WORKDIR}"
```

阶段五: 测试验证 (1-2天)

步骤 5.1: 功能测试

```
# 环境初始化测试
source smart-env

# 工具链安装测试
bitbake smart-gcc -c install_toolchain

# 完整构建测试
bitbake rt-smart -c build_all

# 多架构测试
sed -i 's/qemuarm64/qemuriscv64/' build/conf/local.conf
bitbake rt-smart -c cleanall
bitbake rt-smart -c build_all
```

步骤 5.2: QEMU 启动测试

```
cd build/qemuarm64
./run_qemuarm64.sh
```

阶段六: 文档更新 (1天)

步骤 6.1: 更新使用说明

- 更新 README.md 使用流程
- 创建迁移指南
- 编写故障排除文档

实施时间表

阶段	时间	主要任务	交付物
阶段一	1-2天	环境准备和依赖分析	BitBake集成、依赖分析
阶段二	2-3天	创建 meta-core 层	完整的核心基础层
阶段三	1天	创建环境脚本	smart-env 脚本
阶段四	1-2天	适配现有配方	兼容的配方文件
阶段五	1-2天	集成测试验证	测试报告
阶段六	1天	文档和优化	完整文档

总预期时间：7-11 个工作日

风险评估

主要风险

1. **依赖复杂性**：可能漏掉关键的 poky 依赖组件
2. **兼容性问题**：新环境与现有配方不兼容
3. **维护成本**：需要维护独立的核心层

应对策略

1. 分阶段测试，逐步替换验证
2. 保持向后兼容，渐进式迁移
3. 最小化核心组件，关注必需功能

成功标准

- ☒ 能够在不依赖 poky 的情况下完成完整构建
- ☒ 支持 qemuarm64 和 qemuriscv64 两种架构
- ☒ 构建产物与原方案完全一致
- ☒ 构建时间不超过原方案的 120%
- ☒ 提供完整的使用文档和示例

预期收益

1. **独立性**：完全摆脱对 yocto/poky 的依赖
2. **轻量化**：减少不必要的组件和依赖
3. **可控性**：完全掌控构建环境和流程
4. **扩展性**：更容易添加新的架构和功能
5. **维护性**：简化的架构便于长期维护